

What should I do next?

Given where you are now, I would **not** continue adding plotting functions yet.

The next step is to trace the execution path and identify where the final GLM result actually exists.

Step 1: Find where the GLM is run

Open `main.py` and look for something like:

```
objective1.analyze(...)
objective2.analyze(...)
objective3.analyze(...)
...

```

Python

Run

or similar function calls.

You need to know:

- Is the GLM file actually being executed?
- What function returns the fitted model?
- What variable contains the predictions?

Without that information, a plotting function has nothing to plot.

Step 2: Verify the GLM produces predictions

In the GLM file (e.g. `tweedie_glm.py`), temporarily add:

Python

Run

Run the project.

If you see predicted DC power values in the terminal, the model is producing usable outputs.

Step 3: Save predictions to a DataFrame

You eventually want something like:

```
prediction_df = df.copy()
```

Python

Run

This DataFrame becomes the input to the plotting function.

Step 4: Generate a simple figure first

Before attempting a 3D surface, try a basic check:

```
import matplotlib.pyplot as plt

plt.scatter(
    prediction_df["SPL_dB"],
    prediction_df["predicted"]
)
plt.show()
```

If that works, then:

- the model runs,
- predictions exist,
- plotting works.

Only then move to the 3D surface.

Step 5: Show me the GLM file

The most efficient next action is to paste:

analysis/tweedie_glm.py

(or whichever file currently fits the GLM).

Once I can see that file, I can tell you exactly:

1. Where to generate predictions.
2. Where to call the plotting function.
3. What changes are needed in `main.py` (if any).
4. Whether the figure should be generated automatically during analysis or as a separate post-processing step.

Right now, the bottleneck is not the plotting code—it's identifying where the fitted model exists in your current workflow.



This is the `tweedie_glm.py` file:

```
from __future__ import annotations
```

```
import numpy as np
```

```
import pandas as pd
```

```
from scipy import stats
```

```
import statsmodels.api as sm
```

```
import statsmodels.formula.api as smf
```

```
def tweedie_family():
```

```
    return sm.families.Tweedie(var_power=1.9,
```

Show more ▾

Now we can see exactly why you don't have any visuals.

Your `tweedie_glm.py` currently does **not generate predictions** and does **not call any plotting functions**.

It only returns:

Python

Run

which means the workflow is:

```
Fit Tweedie GLM
↓
Calculate statistics
↓
Return tables
```

There is no:

```
Predict
↓
Plot
↓
Save figure
```

stage.

What should be added?

After:

```
full = smf.glm(
    formula=full_formula,
    data=df,
    family=tweedie_family()
) fit()
```

create predictions:

```
predictions = df.copy()

predictions["predicted"] = full.predict(df)

predictions["residual"] = (
```

```
predictions[variable]  
- predictions["predicted"]  
)
```

This is the missing piece.

Then create a prediction table

I would add:

```
prediction_table = predictions[  
    [  
        "Setup",  
        "Environment",  
        "SPL_dB",  
        variable,  
        "predicted",  
        "residual",  
    ]  
>].copy()
```

For DC Power this becomes:

```
Setup  
Environment  
SPL_dB  
DC_Power  
predicted  
residual
```

Then return it

Instead of:

Python

Run

use:

```
return (  
    model_fit,  
    lr,  
    params,  
    diag,  
    prediction_table,  
    full,  
)
```

where:

`prediction_table`

contains predictions

and

`full`

contains the fitted GLM object.

Why return the model itself?

Because your future 3D plots should be based on:

Python

Run

not on observed medians.

That way the figure represents:

Selected Tweedie GLM

instead of:

Raw data summary

Where should the figure generation occur?

Not inside `tweedie_glm.py`.

Keep this file focused on:

```
Fit model
Generate diagnostics
Generate predictions
Return outputs
```

Then in the file that calls:

Python

Run

you would do:

```
(
    model_fit,
    lr,
    params,
    diag,
    prediction_table,
    fitted_model,
) = fit_tweedie_models(...)
```

and then:

```
plot_dc_power_glm_surface(
    prediction_df=prediction_table,
    setup_label="Standalone",
    out_path=...
)
```

Another issue I see

Your current formula is:

```
full_formula = (
  f"{variable} ~ "
  "C(Environment) * C(SPL_dB)"
)
```

Dutton

Dun

Notice that **Setup** is missing.

Yet your dataset contains:

Standalone

Integrated

(or A/B)

and those setups behave very differently.

For a final thesis model, I would strongly consider:

```
full_formula = (
  f"{variable} ~ "
  "C(Setup)"
  " + C(Environment)"
  " + C(SPL_dB)"
  " + C(Setup):C(Environment)"
  " + C(Setup):C(SPL_dB)"
)
```

or at minimum:

```
full_formula = (
  f"{variable} ~ "
  "C(Setup)"
  " + C(Environment)"
  " + C(SPL_dB)"
)
```

because otherwise the Tweedie model is ignoring one of the most important factors in the experiment.

My recommendation

Before touching the plotting code again:

1. Modify `tweedie_glm.py` to return predictions.
2. Verify `prediction_table.head()` prints correctly.
3. Verify the model includes `Setup`.
4. Identify the file that calls `fit_tweedie_models()`.

Once you show that calling file, we can wire the 3D surfaces into the workflow properly. Right now, the plotting function has no access to predictions because `tweedie_glm.py` never creates or returns them.

       Sources

Ran into an issue with `main.py`. Here is the file for reference:

```
from __future__ import annotations
```

```
import json
```

```
from data.master_dataset import DATA
from utils.validation import load_dataframe,
add_detection_flag
from utils.config import OUTPUT_DIR, FIGURES_DIR,
RESULTS_DIR, TABLES_DIR
```

Show more ▾

The issue is that your `main.py` currently has no knowledge that the Tweedie GLM exists.

It only runs:

```
analyze_objective1(...)
analyze_objective2(...)
```

Python

Run

and collects their outputs.

There is no:

Python

Run

anywhere in the file.

Why you're running into problems

Earlier I suggested returning:

```
(  
    model_fit,  
    lr,  
    params,  
    diag,  
    prediction_table,  
    full,  
)
```

from `fit_tweedie_models()` .

However, `main.py` expects each analysis module to follow a pattern like:

Python

Run

and return a dictionary.

Your Tweedie file currently follows neither pattern.

The clean solution

Create a wrapper function inside `tweedie_glm.py` .

For example:

```
def analyze(df, out_tables, out_figures):
```

Python

Run

Then `main.py` can treat it exactly like Objectives 1–3.

≡ ChatGPT ▾  Free offer

  ...

Near the bottom:

```
from utils.export_helpers import save_table

def analyze(df, out_tables, out_figures):

    model_fit, lr, params, diag = fit_tweedie_models(
        df,
        "DC_Power",
    )

    results = {}

    results["model_fit"] = save_table(
        model_fit,
        out_tables,
        "tweedie_model_fit",
        title="Tweedie Model Fit"
    )

    results["likelihood_ratio"] = save_table(
        lr,
        out_tables,
        "tweedie_lr_tests",
        title="Likelihood Ratio Tests"
    )

    results["parameters"] = save_table(
        params,
        out_tables,
        "tweedie_parameters",
        title="Tweedie Parameters"
    )
```

```
results["diagnostics"] = save_table(  
    diag,  
    out_tables,  
    "tweedie_diagnostics",  
    title="Tweedie Diagnostics"  
)  
  
return results
```

Then modify main.py

Add:

Python

Run

alongside the other imports.

Then update:

```
all_results = {
```

Python

Run

to:

Python

Run

```
all_results = {  
    "objective1": analyze_objective1(  
        df,  
        TABLES_DIR,  
        FIGURES_DIR,  
    ),  
  
    "objective2": analyze_objective2(  
        df,  
        TABLES_DIR,  
        FIGURES_DIR,  
    ),  
}
```

```
"objective3": analyze_objective3(  
    df,  
    TABLES_DIR,  
    FIGURES_DIR,  
),  
  
"tweedie_glm": analyze_tweedie_glm(  
    df,  
    TABLES_DIR,  
    FIGURES_DIR,  
),  
}
```

Before worrying about the 3D surface

I would first confirm that this produces:

```
outputs/  
├─ tables/  
│   ├── tweedie_model_fit.csv  
│   ├── tweedie_lr_tests.csv  
│   ├── tweedie_parameters.csv  
│   └─ tweedie_diagnostics.csv
```

If those files appear, then:

1. The Tweedie module is actually running.
2. `main.py` is connected correctly.
3. We can then add prediction exports.
4. After that, we can wire in the 3D surface generation.

Right now the bigger architectural problem is that `main.py` never executes `tweedie_glm.py`, so no GLM outputs—visual or otherwise—can be produced.

       Sources